به نام خدا



درس داده کاوی- ترم ۴۰۵۱

استاد : دکتر غیور باغبانی

علی کارخانه ۴۰۳۵۲۲۰۵۵

۳ .۲ بخش دوم : شرح مسئله

۳. ۲. ۱ گام اول : شناخت وظایف داده کاوی

۱ .آیا می‌شود فقط از روی اسم مکان (main_value) نوع مکان را حدس زد؟

- وظیفه داده‌کاوی: دسته‌بندی (Classification)
- دلیل: چون هدف این است که با استفاده از نام مکان، آن را در یک دسته یا کلاس مشخص، مانند رستوران، پارک یا موزه قرار دهیم. برای این کار معمولاً به داده‌های برچسب‌گذاری‌شده نیاز داریم تا مدل بتواند از روی نمونه‌های قبلی یاد بگیرد.

۲ .کدام مناطق جغرافیایی تراکم بیشتری از مکان‌های تفریحی دارند؟

- وظیفه داده‌کاوی :خوشه‌بندی (Clustering)
- دلیل : با گروه‌بندی مکان‌های جغرافیایی بر اساس نزدیکی و تراکم آن‌ها می‌توان مناطق پرتراکم را شناسایی کرد . البته اگر مناطق از قبل مشخص باشند و بخواهیم تعداد مکان‌های هر منطقه را محاسبه کنیم، این کار بیشتر تحلیل توصیفی داده‌هاست

۳ .کدام رکوردها نسخه تکراری یک مکان دیگرند.

- وظیفه داده‌کاوی :تشخیص ناهنجاری (Anomaly Detection)، با این توضیح که تشخیص رکوردهای تکراری دقیقاً معادل تشخیص ناهنجاری نیست.
- دلیل :رکوردهای تکراری را می‌توان با مقایسه نام، مختصات جغرافیایی و سایر ویژگی‌ها شناسایی کرد .این مسئله به‌طور دقیق‌تر، تشخیص رکوردهای تکراری (Duplicate Detection)نام دارد و در پاک‌سازی داده‌ها انجام می‌شود.

۳. ۲. ۲ گام دوم : نوع ویژگی و آمار پایه

۱. نوع هر یک از ستون‌های hub_km ، capacity ، lat/lon ، edit_date ، version و main_key را (اسمی، ترتیبی، فاصله‌ای یا نسبتی) مشخص کنید و برای هرکدام یک جمله دلیل بیاورید.

۱ .ستون hub_km: مقیاس نسبتی (Ratio)

این ستون فاصله را بر حسب کیلومتر نشان می‌دهد؛ صفر به معنای نبود فاصله است و مثلاً ۱۰ کیلومتر دو برابر ۵ کیلومتر است.

۲ .ستون capacity : مقیاس نسبتی (Ratio)

اگر این ستون ظرفیت را به‌صورت تعداد یا مقدار قابل‌اندازه‌گیری نشان دهد، صفر به معنای نبود ظرفیت است و نسبت بین مقادیر آن معنا دارد.

۳ .ستون lat/lon : مقیاس فاصله‌ای (Interval)

اختلاف مختصات جغرافیایی معنا دارد، اما صفر در عرض و طول جغرافیایی یک مبدأ قراردادی است و به معنای نبود موقعیت جغرافیایی نیست.

۴ .ستون edit_date : مقیاس فاصله‌ای (Interval)

اختلاف بین تاریخ‌ها قابل‌اندازه‌گیری است؛ مثلاً می‌توان فاصله زمانی بین دو تاریخ را محاسبه کرد، اما مبدأ تاریخ به‌خودی‌خود صفر واقعی محسوب نمی‌شود.

۵ .ستون version : مقیاس ترتیبی (Ordinal)

شماره نسخه‌ها معمولاً ترتیب آن‌ها را مشخص می‌کند، اما فاصله بین نسخه‌ها لزوماً معنای کمّی ندارد.

۶ .ستون main_key : مقیاس اسمی (Nominal)

این ستون برای شناسایی رکوردها استفاده می‌شود و مقادیر آن شناسه هستند؛ بنابراین ترتیب یا فاصله عددی بین آن‌ها معنای خاصی ندارد.

۲. آمار پایه‌ی دیتاست را گزارش دهید :تعداد کل رکوردها، سهم هرکدام از هفت دسته‌ی main_key (amenity, shop, leisure, office, tourism, craft, healthcare) و درصد مقدار گمشده در هر ستون

.کد:

```
Full_data = pd.read_csv("iran_poi_raw.csv")
Total_records = Full_data.shape[0]
print()
print(f'Total records of dataset: {Total_records}')

Categories = [
    "amenity" , "shop" , "leisure" , "office" ,
    "tourism" , "craft" , "healthcare"
]
Counts = Full_data["main_key"].value_counts()
for category in Categories:
    count = Counts.get(category , 0)
    precentage = count / Total_records
    print(f"{category} => count: {count} | precentage : {precentage:.2f}")

missing_count = Full_data.isna().sum()
missing_percentage = (missing_count / Total_records) * 100

missing_report = pd.DataFrame({
    "Missing Values": missing_count,
    "Missing Percentage (%)": missing_percentage.round(2)
})

print("\nMissing Values Report:")
print(missing_report.to_string())
```

در ابتدا، با اســتفاده از تابع pd.read_csv() فایل iran_poi_raw.csv خوانده می‌شـــود و اطلاعات آن در قالب یک DataFrameبه نام Full_data ذخیره می‌شود .سـپس با اسـتفاده از Full_data.shape[0] تعداد کل رکوردهای دیتاسـت محاسبه و در متغیر Total_recordsذخیره می‌شود .در این دستور، عدد صفر نشان‌دهنده تعداد سطرهای دیتافریم است.

در ادامه، لیستی به نام Categories شامل هفت دسته amenity، shop، leisure، office، tourism، craftو healthcare تعریف می‌شـود .سـپس با اسـتفاده از تابع value_counts() تعداد تکرار هر مقدار در سـتون main_key محاسـبه و در متغیر Counts ذخیره می‌شـود .با اسـتفاده از حلقه for، هر یک از دسـته‌های مشـخص‌شـده بررسـی می‌شـوند .دسـتور Counts.get(category, 0)تعداد رکوردهای مربوط به هر دسـته را اسـتخراج می‌کند و اگر دسـته‌ای در داده‌ها وجود نداشـته باشـد، مقدار صـفر را برمی‌گرداند .سـپس با تقسـیم تعداد رکوردهای هر دسـته بر تعداد کل رکوردها، سـهم آن دسـته از کل دیتاسـت محاسـبه می‌شـود و نام دسـته، تعداد رکوردها و سـهم آن به‌صـورت اعشـاری با دو رقم بعد از ممیز نمایش داده می‌شود.

در بخش بعد، با استفاده از Full_data.isna().sum() تعداد مقادیر گمشده در هر ستون محاسبه می‌شود .دستور isna() خانه‌هایی را که مقدار گمشـده دارند مشـخص می‌کند و تابع sum() تعداد آن‌ها را در هر سـتون به دسـت می‌آورد .سـپس درصد مقادیر گمشده هر ستون با تقسیم تعداد مقادیر گمشده بر تعداد کل رکوردها و ضرب نتیجه در ۱۰۰ محاسبه می‌شود.

در نهایت، با اسـتفاده از pd.DataFrame() یک دیتافریم جدید به نام missing_report سـاخته می‌شـود که شـامل تعداد مقادیر گمشـده و درصـد آن‌ها در هر سـتون است .تابع round(2) درصـدها را به دو رقم اعشـار گرد می‌کند .در پایان، گزارش مقادیر گمشـده همراه با عنوان آن نمایش داده می‌شود .استفاده از to_string() باعث می‌شود کل گزارش به‌صورت متنی و جدولی چاپ شود.

خروجی:

```
Total records of dataset: 297152
amenity => count: 138493 | precentage : 0.47
shop => count: 72122 | precentage : 0.24
leisure => count: 42320 | precentage : 0.14
office => count: 25972 | precentage : 0.09
tourism => count: 12220 | precentage : 0.04
craft => count: 3911 | precentage : 0.01
healthcare => count: 2114 | precentage : 0.01

Missing Values Report:
                Missing Values  Missing Percentage (%)
osm_type                     0                    0.00
osm_id                       0                    0.00
version                      0                    0.00
edit_date                    0                    0.00
lat                          0                    0.00
lon                          0                    0.00
main_key                     0                    0.00
main_value                   0                    0.00
name                     73574                   24.76
name:fa                 289881                   97.55
name:en                 251448                   84.62
opening_hours           262518                   88.34
phone                   250662                   84.35
...
n_tags                       0                    0.00
nearest_hub                  0                    0.00
hub_km                       0                    0.00
other_tags              193459                   65.10
```

۳. بعضی ستون‌ها مثل capacity یا cuisine بیش از ۹۵٪ مقدارشان خالی است .توضیح دهید چرا این لزوماً خطای جمع‌آوری داده نیست و چه دلیل دیگری می‌تواند داشته باشد.

زیرا ممکن است آن ویژگی فقط برای نوع خاصی از مکان‌ها کاربرد داشته باشد .برای مثال، ستون cuisineنوع غذای ارائه‌شده را مشخص می‌کند و معمولاً برای رستوران‌ها و مکان‌های مرتبط با غذا معنا دارد، نه برای همه مکان‌ها .همچنین ستون capacityممکن است فقط برای مکان‌هایی ثبت شود که ظرفیت مشخصی دارند یا این اطلاعات درباره آن‌ها در دسترس است .بنابراین، خالی بودن این ستون‌ها می‌تواند ناشی از نامرتبط بودن ویژگی با بسیاری از رکوردها یا اختیاری بودن ثبت اطلاعات باشد، نه لزوماً نقص در فرایند جمع‌آوری داده‌ها.

۳. ۲. ۳ گام سوم: کیفیت داده

۱. ستون addr:city را باز کنید و ببینید اسم شهرها چقدر به‌هم‌ریخته ثبت شده‌اند .تعداد مقادیر یکتای این ستون را گزارش دهید و دست‌کم چهار شکل نوشتاری متفاوت برای یک شهر واقعی پیدا کنید؛ مثلاً «tehran»و «شهر تهران» و «Tehran»و «تهران».

کد:

```
cities = Full_data["addr:city"]

unique_cities = cities.nunique(dropna=True)
missing_cities = cities.isna().sum()

print("Unique city values:", unique_cities)
print("Missing city values:", missing_cities)



print("\nMost frequent city values:")
print(cities.value_counts().head(20).to_string())


tehran_variants = [
    "تهران",
    "Tehran",
    "شهر تهران",
    "tehran",
    "TEHRAN"
]

print("\nWritten forms of Tehran:")

for city in tehran_variants:
    count = (cities == city).sum()
    print(f"{city}: {count}")
```

در این کد، ابتدا ستون addr:cityاز دیتاست استخراج می‌شود و در متغیر citiesقرار می‌گیرد .سپس با استفاده از تابع nunique(dropna=True)تعداد مقادیر یکتای این ستون محاسبه می‌شود؛ به این صورت که مقادیر تکراری و مقادیر گمشده در شمارش لحاظ نمی‌شوند .همچنین با استفاده از isna().sum()تعداد مقادیر گمشده این ستون به دست می‌آید.

در ادامه، تابع value_counts()تعداد تکرار هر نام شهر را محاسبه می‌کند و با استفاده از head(20)، بیست مقدار پرتکرار نمایش داده می‌شوند .این کار کمک می‌کند شکل‌های مختلف ثبت نام شهرها و میزان تکرار آن‌ها بررسی شود.

در بخش آخر، پنج شکل نوشتاری متفاوت برای شهر تهران، شامل تهران، Tehran، شهر تهران، tehranو TEHRAN، در یک لیست قرار می‌گیرند .سپس با استفاده از حلقه forو مقایسه هر مقدار با ستون cities، تعداد تکرار دقیق هر شکل نوشتاری محاسبه و نمایش داده می‌شود.

خروجی:

```
Unique city values: 2525
Missing city values: 252005

Most frequent city values:
addr:city
6110        تهران
4939        شیراز
1194       اصفهان
1125         مشهد
Tehran       1018
932     شهر تهران
932          بابل
835           کرج
821         تبریز
780     چهارفریضه
600      فولادشهر
521           کیش
481         اقلید
479         اهواز
473        اردبیل
452        بروجرد
tehran        408
395       قائمشهر
387      بندرعباس
387           رشت
...
Tehran: 1018
شهر تهران: 932
tehran: 408
TEHRAN: 8
```

۲. ببینید چند نام در ستون nameحرف عربی «ي» یا «ك» دارند، به جای معادل فارسی‌شان «ی» یا «ک» ، و این عدد را گزارش دهید .چرا این نوع نویز مخصوص داده‌ای است که کاربرهای عادی وارد کرده‌اند، نه داده‌ای که یک سازمان با فرمت ثابت تولید کرده؟

کد و خروجی:

```python
arabic_chars = Full_data["name"].str.contains("[يك]", na=False)

total_names = arabic_chars.sum()
unique_names = Full_data.loc[arabic_chars, "name"].nunique()

print("Total records with Arabic characters:", total_names)
print("Unique names with Arabic characters:", unique_names)
```

✓ 0.0s

```
Total records with Arabic characters: 1726
Unique names with Arabic characters: 1697
```

در این کد، ابتدا نام‌هایی که شـامل حداقل یکی از حروف عربی «ي» یا «ك» هسـتند، شـناسـایی می‌شـوند .عبارت [يك] مشخص می‌کند که وجود هرکدام از این دو حرف برای انتخاب یک نام کافی است و na=Falseباعث می‌شود مقادیر گمشده به‌عنوان مقدار Falseدر نظر گرفته شوند.

سـپس با اسـتفاده از arabic_chars.sum()تعداد کل رکوردهایی که نام آن‌ها شـامل این حروف عربی اسـت محاسـبه می‌شـود .در ادامه، دسـتور Full_data.loc[arabic_chars, "name"].nunique()تعداد نام‌های یکتای دارای این حروف را به دست می‌آورد؛ بنابراین نام‌های تکراری فقط یک بار در این شمارش لحاظ می‌شوند .در پایان، تعداد کل رکوردهای دارای حروف عربی و تعداد نام‌های یکتای آن‌ها نمایش داده می‌شود.

این نوع نویز معمولاً در داده‌های واردشـده توسـط کاربران عادی بیشـتر دیده می‌شـود؛ زیرا کاربران ممکن اسـت از صـفحه‌کلیدهای مختلف اسـتفاده کنند یا هنگام تایپ، حروف عربی را به‌جای معادل فارسـی آن‌ها وارد کنند .در مقابل، سازمان‌هایی که داده‌ها را با فرمت ثابت و از طریق سامانه‌های مشخص تولید می‌کنند، معمولاً قواعد یکسانی برای ورود و ثبت اطلاعات دارند و در نتیجه احتمال بروز چنین ناهماهنگی‌هایی کمتر است.

۳. سـتون capacity باید عددی باشـد، ولی نیسـت .مقادیری را که عدد معتبر نیسـتند اسـتخراج و دسـته‌بندی کنید؛ برای مثال عدد منفی، واحد اشـتباه (متر مربع به جای نفر)، یا حتی اسم یک مکان یا شـهر به جای عدد .برای هر دسـته بنویسـید این خطا از کجا می‌تواند آمده باشد.

کد:

```
capacity = Full_data["capacity"].astype("string").str.strip()

missing_value = capacity.isna()

numeric_capacity = pd.to_numeric(capacity, errors="coerce")

invalid = ~missing_value & numeric_capacity.isna()

negative = numeric_capacity < 0


print("Missing capacity values:", missing_value.sum())
print("Valid non-negative numeric values:", (numeric_capacity.notna() & ~negative).sum())
print("Invalid non-numeric values:", invalid.sum())
print("Negative values:", negative.sum())


print("\nNegative capacity values:")
print(Full_data.loc[negative, ["capacity"]].value_counts().to_string())


print("\nInvalid non-numeric values:")
print(Full_data.loc[invalid, ["capacity"]].value_counts().head(10).to_string())
```

در این کد، ابتدا ستون capacityاستخراج می‌شود و با استفاده از astype("string").str.strip() مقادیر آن به رشته تبدیل شـده و فاصله‌های اضافی ابتدا و انتهای متن حذف می‌شوند .سپس با استفاده از isna() تعداد مقادیر گمشـده مشخص می‌شـود .در مرحله بعد، تابع pd.to_numeric(capacity, errors="coerce") مقادیر را به عدد تبدیل می‌کند؛ اگر مقداری قابل تبدیل به عدد نباشد، به NaN تبدیل می‌شود .به این ترتیب، می‌توان مقادیر عددی معتبر را از مقادیر نامعتبر تشخیص داد.

با اســتفاده از شــرط (numeric_capacity.isna() & missing_value~، مقادیری که از ابتدا خالی نبوده‌اند اما به عدد تبدیل نمی‌شوند، شناسایی می‌شوند .همچنین شرط numeric_capacity < 0 برای شناسایی ظرفیت‌های منفی به کار می‌رود .در ادامه، تعداد مقادیر گمشـــده، مقادیر عددی معتبر و غیرمنفی، مقادیر غیرعددی و مقادیر منفی محاســـبه و نمایش داده می‌شـود .در پایان نیز مقادیر منفی و ده مقدار پرتکرار از مقادیر غیرعددی اسـتخراج می‌شـوند تا امکان بررسـی و دسـته‌بندی آن‌ها فراهم شود.

مقادیر نامعتبر ستون capacityرا می‌توان به چند دسته تقسیم کرد:

۱ .اعداد منفی :ظرفیت معمولاً نمی‌تواند منفی باشـد؛ بنابراین این مقادیر ممکن اسـت ناشـی از خطای ورود اطلاعات، اشـتباه در تبدیل داده‌ها یا ثبت نادرست مقدار باشند.

۲ .مقادیر دارای واحد یا متن اضـافی :مقادیری مانند 50 persons یا 200 m^2ممکن اسـت به‌جای یک عدد سـاده ثبت شـده باشند .این مشکل می‌تواند به دلیل ورود اطلاعات به شکل متنی یا استفاده از واحد نامناسب رخ داده باشد.

۳ .نام مکان یا شـهر به‌جای عدد :اگر مقداری مانند نام یک شـهر یا مکان در سـتون ظرفیت ثبت شـده باشـد، احتمال دارد اطلاعات در ستون اشتباه وارد شده باشند یا هنگام ادغام چند منبع داده، مقادیر به‌درستی تطبیق داده نشده باشند.

۴ .متن‌های نامعتبر یا قالب‌های عددی غیرمعمول :عباراتی مانند unknown، خط تیره یا اعدادی با قالبی که مستقیماً توسط تبدیل عددی پشـتیبانی نمی‌شـود، ممکن اسـت به دلیل نامشـخص بودن ظرفیت، تفاوت در شـیوه ثبت داده‌ها یا ناسـازگاری قالب‌ها ایجاد شده باشند.

خروجی:

```
Missing capacity values: 296298
Valid non-negative numeric values: 842
Invalid non-numeric values: 10
Negative values: 2

Negative capacity values:
capacity
-1.0       1
-50.0      1

Invalid non-numeric values:
capacity
۵ 1                    نفر
۵۰                       1
۲۰                       1
۶۰                       1
۱۰                       1
۸۰۰1                متر مربع
1     مخزن (تانکر) آب حسینی
1                      مشهد
1                    ضض300
1                  ترمینال
```

۴. ده نمونه از ستون phoneرا نشان دهید و فرمت‌هایشان را با هم مقایسه کنید :ارقام فارسی در برابر لاتین، با یا بدون 98+، با یا بدون فاصله، و شماره‌هایی که حتی کامل هم نیستند .این فرمت‌ها را دسته‌بندی کنید.

```
phones = Full_data["phone"].astype("string").str.strip()

phones = phones[phones != ""]

persian_digits = "۰۱۲۳۴۵۶۷۸۹"
arabic_digits = "٠١٢٣٤٥٦٧٨٩"

def get_phone_format(phone):
    has_persian = any(ch in persian_digits for ch in phone)
    has_latin = any(ch in "0123456789" for ch in phone)
    has_arabic = any(ch in arabic_digits for ch in phone)

    if has_latin:
        digit_type = "Latin digits"
    elif has_arabic:
        digit_type = "Arabic digits"
    elif has_persian:
        digit_type = "Persian digits"
    else:
        digit_type = "No digits"

    if phone.startswith("+98") or phone.startswith("+۹۸") :
        country_code = "With +98"
    elif phone.startswith("0098") or phone.startswith("۰۰۹۸"):
        country_code = "With 0098"
    elif phone.startswith("98") or phone.startswith("۹۸"):
        country_code = "With 98"
    else:
        country_code = "Without country code"

    spacing = "With spaces" if " " in phone else "Without spaces"
    digits = "".join(
    ch for ch in phone
    if ch in persian_digits + arabic_digits + "0123456789"
)

    if len(digits) < 8:
        length_type = "Possibly incomplete"
    elif 8 < len(digits) < 10:
        length_type = "if it's phone , Incomplete"
    else:
        length_type = "+10 digits"



    return digit_type, country_code, spacing, length_type


phone_formats = phones.apply(get_phone_format)

phone_table = pd.DataFrame({
    "phone": phones,
    "digit_type": [x[0] for x in phone_formats],
    "country_code": [x[1] for x in phone_formats],
    "spacing": [x[2] for x in phone_formats],
    "length_type": [x[3] for x in phone_formats]
})


examples = phone_table.drop_duplicates(
    subset=["digit_type", "country_code", "spacing", "length_type"]
).head(20)

print(examples.to_string(index=False))
```

در این کد، ابتدا ستون phoneاز دیتاست استخراج می‌شود و با استفاده از ()astype("string").str.stripمقادیر آن به رشته تبدیل شـده و فاصـله‌های ابتدا و انتهای متن حذف می‌شـوند .سـپس مقادیر خالی حذف می‌شـوند تا بررسـی فقط روی شماره‌های ثبت‌شده انجام شود.

در ادامه، سـه مجموعه از ارقام فارسـی، عربی و لاتین تعریف می‌شـوند تا نوع ارقام اسـتفاده‌شـده در شـماره تلفن‌ها قابل تشخیص باشد .تابع ()get_phone_formatهر شماره را بررسی می‌کند و آن را بر اساس چهار ویژگی دسته‌بندی می‌کند :نوع ارقام، وجود یا نبود پیش‌شماره کشور، وجود یا نبود فاصله و طول شماره.

برای تشخیص نوع ارقام، بررسی می‌شود که شماره شامل ارقام لاتین، عربی یا فارسی باشد .سپس ابتدای شماره بررسی می‌شود تا مشخص شود که پیش‌شماره‌هایی مانند 98+، 0098یا 98وجود دارند یا خیر .همچنین وجود فاصله در شماره بررسـی می‌شـود .در مرحله بعد، تمام ارقام شـماره اسـتخراج و تعداد آن‌ها محاسـبه می‌شـود تا شـماره‌های احتمالاً ناقص شـناسـایی شـوند .در این کد، شـماره‌هایی که کمتر از ۸ رقم دارند، »احتمالاً ناقص «و شـماره‌های دارای ۹ رقم نیز »ناقص، در صـورت شـماره تلفن بودن «در نظر گرفته می‌شـوند؛ البته این دسـته‌بندی تقریبی اسـت و به‌تنهایی معتبر بودن شـماره را مشخص نمی‌کند.

سـپس با اسـتفاده از phones.apply(get_phone_format)تابع روی تمام شـماره‌ها اجرا می‌شـود و نتیجه هر شـماره در متغیر phone_formatsذخیره می‌شود .در ادامه، یک دیتافریم به نام phone_tableساخته می‌شود که شماره تلفن اصلی و چهار ویژگی تشخیص‌داده‌شده را در ستون‌های جداگانه نگهداری می‌کند.

در پایان، دسـتور ()drop_duplicatesنمونه‌های تکراری را بر اسـاس ترکیب چهار ویژگی حذف می‌کند؛ بنابراین از هر ترکیب متفاوت از نوع ارقام، پیش‌شماره، فاصله و طول شماره فقط یک نمونه باقی می‌ماند .سپس با head(20)حداکثر ۲۰ نمونه نمایش داده می‌شود .با توجه به اینکه سؤال ۱۰ نمونه خواسته است، می‌توان head(20)را به head(10)تغییر داد.

خروجی:

phone	digit_type	country_code	spacing	length_type
+98 21 2611 9186;+98 21 2611 9187	Latin digits	With +98	With spaces	+10 digits
+98 21 64451	Latin digits	With +98	With spaces if it's phone , Incomplete	
+983132205774	Latin digits	With +98	Without spaces	+10 digits
+985134043	Latin digits	With +98	Without spaces if it's phone , Incomplete	
021-91099109	Latin digits	Without country code	Without spaces	+10 digits
555	Latin digits	Without country code	Without spaces	Possibly incomplete
+98 96 550	Latin digits	With +98	With spaces	Possibly incomplete
+۹۸۲۱۲۲۸۷۷۱۲۲	Persian digits	With +98	Without spaces	+10 digits
۶۶۴۰۰۵۰۱ , ۶۶۴۰۷۱۴۰	Persian digits	Without country code	With spaces	+10 digits
+98110	Latin digits	With +98	Without spaces	Possibly incomplete
925545555	Latin digits	Without country code	Without spaces if it's phone , Incomplete	
021 36903516	Latin digits	Without country code	With spaces	+10 digits
982188323021+	Latin digits	With 98	Without spaces	+10 digits
۰۲۱۴۹۲۸۶۰۰۰	Persian digits	Without country code	Without spaces	+10 digits
+۹۸ ۵۵۹۳۳۹۲۳	Persian digits	With +98	With spaces	+10 digits
00982144976811	Latin digits	With 0098	Without spaces	+10 digits
+۹۸۳۱۳۶۶۳۵۰۲۰	Arabic digits	With +98	Without spaces	+10 digits
+۹۸۲۱٤٤٦۹۹۰۳۰	Arabic digits	Without country code	Without spaces	+10 digits
88919862 3	Latin digits	Without country code	With spaces if it's phone , Incomplete	
00989354215943 - 07132343490	Latin digits	With 0098	With spaces	+10 digits

۵. مختصـــات lat/lon را تا ۴ رقم اعشـــار گرد کنید و رکوردهایی پیدا کنید که هم مختصـــاتشـــان یکســـانی دارند و هم main_valueنزدیک است .تعدادشان را گزارش دهید و دست‌کم سه نمونه‌ی واقعی بیاورید که به نظرتان یک مکان را دوبار در نقشه ثبت کرده‌اند .این فهرست را برای گام پنجم نگه دارید.

کد :

```
Full_data["lat_round"] = Full_data["lat"].round(4)
Full_data["lon_round"] = Full_data["lon"].round(4)


duplicate_mask = Full_data.duplicated(
    subset=["lat_round", "lon_round"],
    keep=False
)

duplicate_candidates = Full_data.loc[duplicate_mask].sort_values(
    ["lat_round", "lon_round"]
).copy()


duplicate_groups = duplicate_candidates.groupby(
    ["lat_round", "lon_round"]
)

print("Records with duplicate rounded coordinates:",
      len(duplicate_candidates))

count = 0

for coordinates, group in duplicate_candidates.groupby(
    ["lat_round", "lon_round"]
):
    for value, value_group in group.groupby("main_value"):
        if len(value_group) > 1:
            count += 1

duplicate_data = duplicate_candidates[
    duplicate_candidates.duplicated(
        subset=["lat_round", "lon_round", "main_value"],
        keep=False
    )
].copy()

duplicate_data = duplicate_data.sort_values(
    ["lat_round", "lon_round", "main_value"]
)


print("Number of groups with duplicate cordinate and main_value:", count)

columns_to_show = [
    "lat_round",
    "lon_round",
    "name",
    "name:fa",
    "main_key",
    "main_value",
    "addr:city",
    "osm_id"
]


count = 0

for coordinates, group in duplicate_groups:
    for value, value_group in group.groupby("main_value"):
            if len(value_group) > 1:
                print("=" * 80)
                print("Rounded coordinates:", coordinates)
                print("Main value:", value)
```

```
                print(
                    value_group[columns_to_show]
                    .head(10)
                    .to_string(index=False)
                )

                count = count + 1

                if count == 3:
                    break

    if count == 3:
        break
```

در این کد، ابتدا مختصات جغرافیایی ستون‌های lat و lon با استفاده از تابع round(4) تا چهار رقم اعشار گرد می‌شوند و در ستون‌های جدید lat_round و lon_round ذخیره می‌شوند .این کار باعث می‌شود مختصاتی که اختلاف بسیار کمی دارند، پس از گرد کردن یکسان شوند.

سپس با استفاده از duplicated() رکوردهایی شناسایی می‌شوند که مختصات گردشده یکسانی دارند .آرگومان keep=False باعث می‌شود تمام رکوردهای متعلق به مختصات تکراری انتخاب شوند، نه فقط رکوردهای بعدی .این رکوردها در متغیر duplicate_candidates ذخیره و بر اساس مختصات مرتب می‌شوند.

در مرحله بعد، رکوردها با استفاده از groupby() بر اساس مختصات گردشده گروه‌بندی می‌شوند .سپس در هر گروه، مقادیر ستون main_value بررسی می‌شوند تا مشخص شود آیا چند رکورد با مختصات و مقدار یکسان وجود دارند یا خیر .تعداد این گروه‌ها در متغیر count محاسبه می‌شود .همچنین duplicate_data شامل رکوردهایی است که ترکیب lat_round، lon_round و main_value آن‌ها تکراری است.

در پایان، ستون‌های مربوط به مختصات، نام مکان، نام فارسی، دسته‌بندی، شهر و شناسه‌ی osm_id نمایش داده می‌شوند تا بتوان رکوردهای مشکوک به ثبت تکراری را بررسی کرد .حلقه‌ها حداکثر سه گروه از نمونه‌های تکراری را نمایش می‌دهند.

خروجی:

```
Records with duplicate rounded coordinates: 22300
Number of groups with duplicate cordinate and main_value: 3245
================================================================================
Rounded coordinates: (25.2832, 60.6974)
Main value: place_of_worship
 lat_round  lon_round           name name:fa main_key       main_value addr:city     osm_id
   25.2832    60.6974 مسجد امام حسین      NaN  amenity place_of_worship       NaN 7497107222
   25.2832    60.6974            NaN      NaN  amenity place_of_worship       NaN  917649228
================================================================================
Rounded coordinates: (25.2895, 60.6375)
Main value: hotel
 lat_round  lon_round      name name:fa main_key main_value addr:city     osm_id
   25.2895    60.6375 هتل سپیده      NaN  tourism      hotel       NaN 4757824521
   25.2895    60.6375 هتل سپیده      NaN  tourism      hotel       NaN 1137535985
================================================================================
Rounded coordinates: (25.2898, 60.6335)
Main value: park
 lat_round  lon_round name name:fa main_key main_value addr:city     osm_id
   25.2898    60.6335 پارک      NaN  leisure       park       NaN 7436803557
   25.2898    60.6335  NaN      NaN  leisure       park       NaN  880499859
```

۳. ۲. ۴ گام چهارم : پیش پردازش و پاکسازی

۱.ستون شهر (addr:city) را نرمال‌سازی کنید :حروف بزرگ و کوچک لاتین را یکسان کنید، پیشوند (شهر) را حذف کنید و نام لاتین حداقل ۱۰ شهر پرتکرار را به معادل فارسی آن نگاشت کنید .تعداد مقادیر یکتا را قبل و بعد از این عملیات مقایسه کنید.

کد:

```
unique_before = Full_data["addr:city"].nunique()

Full_data["city_normalized"] = (
    Full_data["addr:city"]
    .astype("string")
    .str.strip()
    .str.lower()
)

Full_data["city_normalized"] = Full_data["city_normalized"].str.replace("شهر ", "")

city_mapping = {
    "tehran": "تهران",
    "mashhad": "مشهد",
    "isfahan": "اصفهان",
    "esfahan": "اصفهان",
    "shiraz": "شیراز",
    "tabriz": "تبریز",
    "karaj": "کرج",
    "qom": "قم",
    "ahvaz": "اهواز",
    "ahwaz": "اهواز",
    "kermanshah": "کرمانشاه",
    "urmia": "ارومیه",
    "orumiyeh": "ارومیه",
    "rasht": "رشت",
    "zahedan": "زاهدان",
    "yazd": "یزد",
    "arak": "اراک",
    "bandar abbas": "بندرعباس",
    "hamedan": "همدان",
    "babol": "بابل",
    "ardebil" : "اردبیل",
    "ardabil" : "اردبیل",
    "kish" : "کیش"
}

Full_data["city_normalized"] = Full_data["city_normalized"].replace(
    city_mapping
)

unique_after = Full_data["city_normalized"].nunique()

print("Unique cities before normalization:", unique_before)
print("Unique cities after normalization:", unique_after)
print("Reduction in unique values:", unique_before - unique_after)
print(f"Most frequent cities:")
print(Full_data["city_normalized"].value_counts().head(30).to_string())
```

در این کد، ابتدا تعداد مقادیر یکتای ستون addr:city قبل از نرمال‌سازی با استفاده از تابع nunique() محاسبه و در متغیر unique_before ذخیره می‌شود .سپس مقادیر این ستون به رشته تبدیل شده و با استفاده از str.strip() فاصله‌های ابتدا و انتهای نام شهرها حذف می‌شوند و تابع str.lower() حروف لاتین بزرگ و کوچک را یکسان می‌کند .نتیجه در ستون جدید city_normalizedذخیره می‌شود.

در مرحله بعدعبارت (شهر) از نام شهرها حذف می‌شود تا نام‌ها یکدست‌تر شوند .سپس یک دیکشنری به نام city_mappingتعریف می‌شود که نام لاتین شهرهای پرتکرار را به معادل فارسی آن‌ها تبدیل می‌کند؛ برای مثال، tehranبه «تهران» و mashhadبه «مشهد» نگاشت می‌شود .همچنین نام‌های متفاوت یک شهر، مانند isfahanو esfahan، به یک نام فارسی یکسان تبدیل می‌شوند.

در ادامه، تعداد مقادیر یکتای ستون city_normalizedپس از نرمال‌سازی با استفاده از nunique()محاسبه و در متغیر unique_afterذخیره می‌شود .سپس تعداد مقادیر یکتا قبل و بعد از نرمال‌سازی و میزان کاهش آن‌ها با استفاده از دستورهای print()نمایش داده می‌شود .در پایان نیز با استفاده از value_counts()، تعداد تکرار هر شهر محاسبه شده و ۳۰ شهر پرتکرار نمایش داده می‌شوند تا نتیجه نرمال‌سازی قابل بررسی باشد.

خروجی:

```
Unique cities before normalization: 2525
Unique cities after normalization: 2291
Reduction in unique values: 234
Most frequent cities:
city_normalized
8476        تهران
5067        شیراز
1593         مشهد
1271       اصفهان
983          بابل
947         تبریز
885           کرج
780     چهارفریضه
600      فولادشهر
549         اهواز
523          کیش
505           قم
489       اردبیل
481        اقلید
472          یزد
470          رشت
453       بروجرد
408      بندرعباس
402      قائمشهر
397       ارومیه
...
214         مراغه
213          ساری
211         کرمان
208         کاشان
```

۲. تابعی بنویسید که مقدار capacityرا به عدد صحیح تبدیل کند؛ از جمله ارقام فارسی را به لاتین تبدیل کند .هر مقداری که از نظر منطقی قابل تبدیل نیست، به NaN تبدیل شود و تعداد رکوردهایی که به این صورت NaN شده‌اند، گزارش داده شود.

کد و خروجی:

```
def convert_capacity(value):
    if pd.isna(value):
        return float("Nan")

    value = str(value).strip()

    persian_digits = "۰۱۲۳۴۵۶۷۸۹"
    english_digits = "0123456789"

    for i in range(10):
        value = value.replace(persian_digits[i], english_digits[i])

    try:
        return int(value)
    except ValueError:
        return float("nan")


Full_data["capacity_numeric"] = Full_data["capacity"].apply(convert_capacity)

nan_count = Full_data["capacity_numeric"].isna().sum()

print("Number of NaN records:", nan_count)
```

```
Number of NaN records: 296588
```

در این کد، ابتدا تابع convert_capacity()تعریف می‌شود تا مقادیر ستون capacityرا به عدد صحیح تبدیل کند .اگر مقدار ورودی خالی یا NaNباشد، تابع مقدار NaNبرمی‌گرداند؛ در غیر این صورت، مقدار به رشته تبدیل شده و فاصله‌های ابتدا و انتهای آن با strip()حذف می‌شوند.

سپس دو رشته شامل ارقام فارسی و لاتین تعریف می‌شوند و با استفاده از حلقه for، هر رقم فارسی با رقم لاتین متناظر خود جایگزین می‌شود .برای مثال، عدد ۱۲۵ به 125تبدیل می‌شود.

در مرحله بعد، دستور int(value)تلاش می‌کند مقدار را به عدد صحیح تبدیل کند .اگر مقدار قابل تبدیل نباشد، مثلاً شامل حروف یا متن نامعتبر باشد، خطای ValueError رخ می‌دهد و بخش exceptمقدار NaN را برمی‌گرداند.

در ادامه، تابع با استفاده از apply(convert_capacity) روی تمام مقادیر ستون capacityاجرا می‌شود و نتیجه در ستون جدید capacity_numeric ذخیره می‌شود .در پایان، با دستور isna().sum() تعداد مقادیری که پس از تبدیل NaN شده‌اند محاسبه و با نمایش داده می‌شود.

۳ . ارقام فارسی و لاتین ستون phone را یکدست کنید (همه به لاتین) و یک قاعده‌ی ساده برای شماره‌ی تلفن تعریف کنید، مثلاً کمتر از ۸ رقم .گزارش دهید چند مورد این قاعده‌ی »شماره‌ی احتمالاً نامعتبر « را نقض می‌کنند.

کد و خروجی:

```
Full_data["phone_normalized"] = Full_data["phone"].astype("string")

persian_digits = "۰۱۲۳۴۵۶۷۸۹"
english_digits = "0123456789"

for i in range(10):
    Full_data["phone_normalized"] = Full_data["phone_normalized"].str.replace(
        persian_digits[i], english_digits[i]
    )

digit_count = Full_data["phone_normalized"].str.count(r"\d")


invalid_phone = (
    Full_data["phone_normalized"].notna()
    & Full_data["phone_normalized"].str.strip().ne("")
    & (digit_count < 8)
)

print("Number of probably invalid phone numbers:", invalid_phone.sum())
```

```
Number of probably invalid phone numbers: 127
```

در این کد، ابتدا یک ستون جدید به نام phone_normalized ایجاد می‌کنیم تا مقدارهای ستون phoneرا در آن نگه داریم . سپس با استفاده از دو رشته‌ی persian_digitsو english_digits، ارقام فارسی را با ارقام لاتین جایگزین می‌کنیم تا شماره‌ها یکدست شوند.

در مرحله‌ی بعد، با استفاده از تابع str.count(r"\d")تعداد ارقام موجود در هر شماره تلفن را محاسبه می‌کنیم و نتیجه را در متغیر digit_countقرار می‌دهیم .سپس یک قاعده‌ی ساده تعریف می‌کنیم :شماره تلفنی که مقدار داشته باشد و کمتر از ۸ رقم داشته باشد، احتمالاً نامعتبر است .برای این کار، شرط notna()مقدارهای خالی از نوع NaNرا حذف می‌کند، شرط str.strip().ne("")رشته‌های خالی یا دارای فاصله را کنار می‌گذارد و شرط digit_count < 8شماره‌هایی را که کمتر از ۸ رقم دارند مشخص می‌کند.

در پایان، با اجرای invalid_phone.sum()تعداد شماره تلفن‌هایی که این قاعده را نقض کرده‌اند محاسبه و چاپ می‌شود .

۳. ۲. ۵ گام پنجم: شباهت و فاصله

۱. از بین کاندیدهای رکورد تکراری که در گام سوم پیدا کردید، حداقل ۲۰ جفتی را انتخاب کنید که هر دو عضوشان ستون name پر دارند. برای هر جفت، شباهت Jaccard را بین مجموعه‌کلیدهای other_tags دو عضو حساب کنید. فرمول Jaccard را خودتان با عملیات مجموعه در پایتون بنویسید؛ استفاده از تابع آماده‌ی کتابخانه برای همین محاسبه پذیرفته نیست.

۲. حالا نام‌های همان جفت‌ها را هم با همین فرمول Jaccard مقایسه کنید، با این تفاوت که این‌بار به‌جای مجموعه‌کلیدهای تگ، مجموعه‌ی کلمات هر نام را می‌سازید؛ یعنی هر نام را از روی فاصله به کلمه‌هایش می‌شکنید (مثلاً «بانک ملی» می‌شود مجموعه‌ی دوعضوی «بانک» و «ملی») و شباهت را روی همین دو مجموعه حساب می‌کنید.

کد:

```
import json

named_data = duplicate_data[
    duplicate_data["name"].notna()
    & duplicate_data["name"].astype("string").str.strip().ne("")
    & ~duplicate_data["name"].astype("string").str.contains(r"\d", na=False)
].copy()



def get_tag_keys(tags):
    if pd.isna(tags):
        return set()

    try:
        data = json.loads(tags)
        return set(data.keys())
    except (ValueError, TypeError):
        return set()


def get_name_set(name):
    if pd.isna(name):
        return set()

    return set(str(name).lower().split())


def jaccard(set1, set2):
    if len(set1 | set2) == 0:
        return 0

    return len(set1 & set2) / len(set1 | set2)


pairs = []

for group_key, group in named_data.groupby(
    ["lat_round", "lon_round", "main_value"]
):
    group = group.reset_index(drop=True)


    for i in range(len(group)):
        for j in range(i + 1, len(group)):

            row1 = group.iloc[i]
            row2 = group.iloc[j]
```

```
            tags1 = get_tag_keys(row1["other_tags"])
            tags2 = get_tag_keys(row2["other_tags"])

            name_similarity = jaccard(
                get_name_set(row1["name"]),
                get_name_set(row2["name"])
            )

            tags_similarity = jaccard(tags1, tags2)

            pairs.append({
                "osm_id_1": row1["osm_id"],
                "osm_id_2": row2["osm_id"],
                "name_1": row1["name"],
                "name_2": row2["name"],
                "lat_1": row1["lat_round"],
                "lon_1": row1["lon_round"],
                "lat_2": row2["lat_round"],
                "lon_2": row2["lon_round"],
                "other_tags_jaccard": tags_similarity,
                "name_jaccard": name_similarity
            })


pairs_data = pd.DataFrame(pairs)


# top_pairs = pairs_data.sort_values(
#     "other_tags_jaccard",
#     ascending=True
# ).head(20)
top_pairs = pairs_data.sample(n=20 , random_state=10)

print("Number of records with names:", len(named_data))
print("Number of pairs:", len(pairs_data))
for index, row in top_pairs.reset_index(drop=True).iterrows():
    print("\n" + "=" * 80)
    print("Pair number:", index + 1)
    print("-" * 80)
    print("OSM ID 1:", row["osm_id_1"])
    print("Name 1:", row["name_1"])
    print("Coordinates 1:",f"({row['lat_1']} , {row['lon_1']})")
    print("OSM ID 2:", row["osm_id_2"])
    print("Name 2:", row["name_2"])
    print("Coordinates 2:", f"({row['lat_2']} , {row['lon_2']})")
    print("Other tags Jaccard:", row["other_tags_jaccard"])
    print("Name Jaccard:", row["name_jaccard"])

print("\n" + "=" * 80)
```

در این کد، ابتدا با استفاده از شرط‌های notna()، str.strip().ne("") و str.contains(r"\d", na=False) رکوردهایی را انتخاب می‌کنیم که ستون name آن‌ها خالی نباشد، فقط شامل فاصله نباشد و هیچ رقم عددی در نام آن‌ها وجود نداشته باشد. نتیجه در متغیر named_data ذخیره می‌شود. استفاده از copy() نیز باعث می‌شود یک کپی مستقل از داده‌های انتخاب‌شده ایجاد شود.

در مرحله‌ی بعد، تابع get_tag_keys(tags) را تعریف می‌کنیم تا کلیدهای موجود در ستون other_tags را استخراج کند. این تابع ابتدا با pd.isna(tags) بررسی می‌کند که مقدار خالی نباشد. سپس با استفاده از json.loads(tags) متن JSON را به

داده‌ی پایتونی تبدیل می‌کند و با set(data.keys()) مجموعه‌ی کلیدهای آن را برمی‌گرداند .اگر مقدار قابل تبدیل نباشد یا خطایی از نوع ValueErrorیا TypeError رخ دهد، مجموعه‌ی خالی برگردانده می‌شود.

سپس تابع get_name_set(name)را تعریف می‌کنیم تا نام هر رکورد را به مجموعه‌ای از کلمات تبدیل کند .در این تابع، ابتدا با str(name).lower()حروف نام را به حروف کوچک تبدیل می‌کنیم و سپس با split() نام را بر اساس فاصله به کلمات تقسیم می‌کنیم .در نهایت، با set() مجموعه‌ی کلمات را می‌سازیم تا کلمات تکراری فقط یک بار در مجموعه قرار بگیرند.

در ادامه، تابع jaccard(set1, set2) را برای محاسبه‌ی ضریب جاکارد دو مجموعه تعریف می‌کنیم .با استفاده از set1 & set2 اشتراک دو مجموعه و با set1 | set2 اجتماع آن‌ها را به دست می‌آوریم .سپس تعداد اعضای اشتراک را بر تعداد اعضای اجتماع تقسیم می‌کنیم .اگر اجتماع دو مجموعه خالی باشد، مقدار صفر برگردانده می‌شود .ضریب جاکارد مقداری بین صفر و یک دارد و هرچه به یک نزدیک‌تر باشد، شباهت دو مجموعه بیشتر است.

پس از تعریف توابع، یک لیست خالی به نام pairs ایجاد می‌کنیم تا اطلاعات جفت رکوردها در آن ذخیره شود .سپس با استفاده از groupby(["lat_round", "lon_round", "main_value"]) رکوردها را بر اساس عرض جغرافیایی گردشده، طول جغرافیایی گردشده و مقدار اصلی گروه‌بندی می‌کنیم .در نتیجه، رکوردهایی که این سه مقدار یکسان دارند، در یک گروه قرار می‌گیرند .برای هر گروه، با استفاده از reset_index(drop=True) اندیس‌ها را از صفر مرتب می‌کنیم.

در مرحله‌ی بعد، دو حلقه‌ی تو در تو با متغیرهای i و jتمام جفت‌های ممکن در هر گروه را بررسی می‌کنند .حلقه‌ی بیرونی از اولین رکورد شروع می‌شود و حلقه‌ی داخلی از رکورد بعدی آغاز می‌شود .به این ترتیب، هر جفت فقط یک بار بررسی می‌شود و هیچ رکوردی با خودش مقایسه نمی‌شود .در هر تکرار، با استفاده از group.iloc[i] و group.iloc[j] اطلاعات دو رکورد را در متغیرهای row1و row2قرار می‌دهیم.

سپس با فراخوانی تابع get_tag_keys()، مجموعه‌ی کلیدهای other_tags هر دو رکورد را استخراج می‌کنیم .همچنین با استفاده از تابع get_name_set()نام هر رکورد را به مجموعه‌ای از کلمات تبدیل می‌کنیم .در ادامه، تابع jaccard() دو بار اجرا می‌شود تا شباهت نام‌ها در متغیر name_similarity و شباهت کلیدهای تگ در متغیر tags_similarityمحاسبه شود.

پس از محاسبه‌ی شباهت‌ها، با استفاده از pairs.append({...}) اطلاعات هر جفت شامل شناسه‌های OSM، نام‌ها، مختصات گردشده و مقادیر شباهت جاکارد در قالب یک دیکشنری به لیست pairsاضافه می‌شود .پس از پایان حلقه‌ها، دستور pd.DataFrame(pairs) این لیست را به یک جدول به نام pairs_data تبدیل می‌کند که هر سطر آن نشان‌دهنده‌ی یک جفت رکورد و اطلاعات مربوط به آن است.

در مرحله‌ی بعد، دستور pairs_data.sample(n=20, random_state=10) بیست جفت را به‌صورت تصادفی انتخاب می‌کند و در متغیر top_pairs قرار می‌دهد .پارامتر random_state=10 باعث می‌شود که در صورت ثابت بودن داده‌ها، انتخاب تصادفی قابل تکرار باشد .توجه داشته باشید که این روش الزاماً جفت‌هایی با شباهت جاکارد بالا انتخاب نمی‌کند؛ بلکه صرفاً بیست جفت تصادفی را برمی‌گزیند .بخش کامنت‌شده‌ی sort_values() و head(20) روش دیگری برای انتخاب جفت‌هاست که در شکل فعلی، بیست جفت با کمترین شباهت تگ‌ها را انتخاب می‌کند، نه بیشترین شباهت.

در ادامه، تعداد رکوردهای دارای نام و تعداد کل جفت‌های تولیدشده نمایش داده می‌شود .این اطلاعات به ما کمک می‌کنند تا تعداد رکوردهای بررسی‌شده و تعداد جفت‌های حاصل از مقایسه را مشاهده کنیم.

در پایان، حلقه‌ی for index, row in top_pairs.reset_index(drop=True).iterrows() اطلاعات بیست جفت انتخاب‌شده را نمایش می‌دهد .ابتدا شماره‌ی هر جفت و یک خط جداکننده چاپ می‌شود .سپس شناسه‌ی OSM، نام و مختصات رکورد اول و دوم نمایش داده می‌شوند .در نهایت، مقدار other_tags_jaccard برای نشان دادن شباهت کلیدهای تگ‌ها و مقدار name_jaccard برای نشان دادن شباهت کلمات نام‌ها چاپ می‌شود .آخرین دستور نیز یک خط جداکننده برای مشخص کردن پایان خروجی نمایش می‌دهد.

خروجی:

```
Number of records with names: 4112
Number of pairs: 2291

============================================================
Pair number: 1
------------------------------------------------------------
OSM ID 1: 2050321983
Name 1: تاکسی ترمینال صفه
Coordinates 1: (32.5891 , 51.6707)
OSM ID 2: 962338208
Name 2: تاکسی ترمینال صفه
Coordinates 2: (32.5891 , 51.6707)
Other tags Jaccard: 0.0
Name Jaccard: 1.0

============================================================
Pair number: 2
------------------------------------------------------------
OSM ID 1: 12064525467
Name 1: نمایندگی کاشی مرجان
Coordinates 1: (29.6255 , 52.514)
OSM ID 2: 13318854866
Name 2: فروشگاه کاشی سرامیک نجیب زاده
Coordinates 2: (29.6255 , 52.514)
Other tags Jaccard: 1.0
```

آیا جفت‌هایی که شباهت تگ‌شان بالاست، شباهت نام‌شان هم بالاست؟ یک جفت پیدا کنید که این دو معیار درباره‌اش توافق ندارند (مثلاً تگ‌ها شبیه‌اند ولی نام‌ها فرق دارند، یا برعکس) و دلیلش را بررسی کنید.

```
============================================================
Pair number: 20
------------------------------------------------------------
OSM ID 1: 11012400759
Name 1: مبل آمین
Coordinates 1: (29.5777 , 52.5045)
OSM ID 2: 11012400760
Name 2: مکعب
Coordinates 2: (29.5777 , 52.5045)
Other tags Jaccard: 1.0
Name Jaccard: 0.0
```

مبل آمین : "{""check_date"": ""2023-06-28""}"

مکعب : "{""check_date"": ""2023-06-28""}"

در این مثال check_date جفت شان برابر است ولی نام متفاوت دارند .

۳ .بر اساس این دو معیار، آستانه‌ای پیشنهاد دهید که بالاترین شانس دو رکورد را «به احتمال زیاد یک مکان واقعی» حساب کند. با این آستانه چند جفت را در نهایت تکراری علامت می‌زنید؟

کد :

```
name_threshold = 0.6
tags_threshold = 0.6



duplicate_pairs = pairs_data[
    (pairs_data["name_jaccard"] >= name_threshold)
    & (pairs_data["other_tags_jaccard"] >= tags_threshold)
]

print("Number of duplicate pairs:", len(duplicate_pairs))

duplicate_pairs = duplicate_pairs.sort_values(
    "other_tags_jaccard",
    ascending=True
)
duplicate_pairs = duplicate_pairs.sort_values(
    "name_jaccard",
    ascending=True
)


for index, row in duplicate_pairs.reset_index(drop=True).iterrows():
    print("=" * 80)
    print("Pair number:", index + 1)
    print("" + "=" * 80)
    print("Name 1:", row["name_1"])
    print("Name 2:", row["name_2"])
    print("Coordinates 1:", row["lat_1"], row["lon_1"])
    print("Coordinates 2:", row["lat_2"], row["lon_2"])
    print("Name Jaccard:", row["name_jaccard"])
    print("Other tags Jaccard:", row["other_tags_jaccard"])
```

در این کد، ابتدا دو آستانه برای ضریب جاکارد تعریف می‌کنیم .مقدار name_threshold = 0.6حداقل شباهت نام‌ها و مقدار tags_threshold = 0.6حداقل شباهت کلیدهای تگ را مشخص می‌کند .سپس با استفاده از این دو آستانه، رکوردهایی را بررسی می‌کنیم که مقدار name_jaccard وother_tags_jaccardآن‌ها هر دو بزرگ‌تر یا مساوی ۰/۶ باشند .این شرط با استفاده از عملگر &برقرار می‌شود؛ بنابراین، هر جفت باید هم از نظر نام و هم از نظر تگ‌ها حداقل شباهت ۶۰ درصدی داشته باشد تا به‌عنوان جفت تکراری انتخاب شود .نتیجه در متغیر duplicate_pairsذخیره می‌شود.

در مرحله‌ی بعد، با دستور print("Number of duplicate pairs:", len(duplicate_pairs))تعداد جفت‌هایی را که هر دو شرط را برآورده کرده‌اند، محاسبه و نمایش می‌دهیم .تابع len()تعداد سطرهای جدول duplicate_pairsرا مشخص می‌کند؛

بنابراین، عدد نمایش‌داده‌شده نشان‌دهنده‌ی تعداد جفت‌هایی است که بر اساس آستانه‌های انتخاب‌شده، نامزد تکراری بودن هستند.

سپس با استفاده از sort_values()، جفت‌ها را ابتدا بر اساس ستون other_tags_jaccard و بعد بر اساس ستون name_jaccard مرتب می‌کنیم. در هر دو دستور، مقدار ascending=True باعث مرتب‌سازی صعودی می‌شود. چون دستور دوم روی نتیجه‌ی دستور اول اجرا می‌شود، مرتب‌سازی نهایی بر اساس شباهت نام‌ها خواهد بود و در صورت برابر بودن شباهت نام‌ها، ترتیب قبلی بر اساس شباهت تگ‌ها حفظ می‌شود.

در پایان، حلقه‌ی for index, row in duplicate_pairs.reset_index(drop=True).iterrows() اطلاعات جفت‌های انتخاب‌شده را یکی‌یکی نمایش می‌دهد. ابتدا شماره‌ی هر جفت و خط جداکننده چاپ می‌شود. سپس نام هر دو رکورد، مختصات آن‌ها، ضریب جاکارد نام‌ها و ضریب جاکارد کلیدهای تگ نمایش داده می‌شود. این اطلاعات به ما کمک می‌کنند تا جفت‌های انتخاب‌شده را بررسی کنیم و تشخیص دهیم که آیا واقعاً به یک مکان اشاره دارند یا صرفاً از نظر نام و تگ شباهت زیادی دارند.

۳. ۲. ۶ گام ششم : آمار توصیفی، تجسم و OLAP

۱. فراوانی و مد main_value را درون هر main_key گزارش دهید؛ مثلاً پرتکرارترین نوع amenity کدام است؟

کد :

```
frequency = Full_data.groupby(["main_key", "main_value"]).size()

for key in Full_data["main_key"].unique():
    data = frequency.loc[key]
    print("=" * 80)
    print("Main key:", key)
    print("Mode:", data.idxmax())
    print("Mode frequency:", data.max())
```

در این کد، ابتدا با استفاده از Full_data.groupby(["main_key", "main_value"]).size() فراوانی هر ترکیب از main_key و main_value را محاسبه می‌کنیم. تابع groupby() داده‌ها را بر اساس این دو ستون گروه‌بندی می‌کند و تابع size() تعداد رکوردهای موجود در هر گروه را به دست می‌آورد. نتیجه در متغیر frequency ذخیره می‌شود؛ بنابراین، می‌توانیم تعداد تکرار هر مقدار از main_value را درون هر main_key بررسی کنیم.

در مرحله‌ی بعد، با استفاده از Full_data["main_key"].unique() تمام مقادیر یکتای ستون main_key را استخراج می‌کنیم تا هر کلید فقط یک بار بررسی شود. سپس حلقه‌ی for روی این مقادیر اجرا می‌شود و با دستور frequency.loc[key]، فراوانی مقادیر مختلف main_value مربوط به همان main_key را استخراج می‌کنیم.

برای محاسبه‌ی مد، از دستور data.idxmax() استفاده می‌کنیم. این دستور مقداری از main_value را برمی‌گرداند که بیشترین فراوانی را در میان مقادیر مربوط به main_key فعلی دارد. سپس با استفاده از data.max()، تعداد تکرار همان مقدار پرتکرار را محاسبه می‌کنیم. در نهایت، مقدار پرتکرار و تعداد تکرار آن را نمایش می‌دهند.

خروجی:

```
============================================================
Main key: amenity
Mode: place_of_worship
Mode frequency: 17810
============================================================
Main key: office
Mode: government
Mode frequency: 9389
============================================================
Main key: shop
Mode: supermarket
Mode frequency: 7667
============================================================
Main key: tourism
Mode: hotel
Mode frequency: 2577
============================================================
Main key: leisure
Mode: park
Mode frequency: 20097
============================================================
Main key: healthcare
Mode: laboratory
Mode frequency: 957
============================================================
Main key: craft
Mode: photographer
Mode frequency: 366
```

۲ .صدک‌های ۲۵، ۵۰، ۷۵ و ۹۰ ستون km_hub را حساب کنید. چند درصد رکوردها بیش از ۳۰۰ کیلومتر از نزدیک‌ترین مرکز استانی تقریبی‌شان فاصله دارند، و این مکان‌ها بیشتر کجای کشورند؟

کد :

```python
hub_data = Full_data["hub_km"].dropna()
print(len(hub_data))

count = (hub_data > 300).sum()
percentage = count / len(hub_data) * 100

print("Number of records over 300 km:", count)
print("Percentage over 300 km:", f"{percentage} %")



hub_data = sorted(hub_data)

n = len(hub_data)

p25 = hub_data[int(0.25 * (n - 1))]
p50 = hub_data[int(0.50 * (n - 1))]
p75 = hub_data[int(0.75 * (n - 1))]
p90 = hub_data[int(0.90 * (n - 1))]

print("\nPercentiles of hub_km:")
print("25th percentile:", p25)
print("50th percentile:", p50)
print("75th percentile:", p75)
print("90th percentile:", p90)

far_places = Full_data[Full_data["hub_km"] > 300]

print("\nMost common cities for records over 300 km:")
print(far_places["nearest_hub"].value_counts().head(10).to_string())

print("\nAverage coordinates of records over 300 km:")
print("Latitude:", far_places["lat"].mean())
print("Longitude:", far_places["lon"].mean())
```

در این کد، ابتدا با استفاده از Full_data["hub_km"].dropna() مقادیر خالی ستون hub_km را حذف می‌کنیم و داده‌های معتبر را در متغیر hub_data ذخیره می‌کنیم .این ستون فاصله‌ی هر رکورد تا نزدیک‌ترین مرکز استانی تقریبی را بر حسب کیلومتر نشان می‌دهد .سپس با دستور print(len(hub_data)) تعداد رکوردهایی را که مقدار فاصله‌ی معتبر دارند، نمایش می‌دهیم.

در مرحله‌ی بعد، با استفاده از شرط hub_data > 300 رکوردهایی را مشخص می‌کنیم که فاصله‌ی آن‌ها تا نزدیک‌ترین مرکز استانی بیشتر از ۳۰۰ کیلومتر است .دستور (hub_data > 300).sum() تعداد این رکوردها را محاسبه می‌کند و نتیجه در متغیر count قرار می‌گیرد .سپس با تقسیم این تعداد بر کل رکوردهای دارای فاصله‌ی معتبر و ضرب آن در ۱۰۰، درصد رکوردهایی را که بیش از ۳۰۰ کیلومتر فاصله دارند، در متغیر percentage محاسبه می‌کنیم .در پایان، تعداد و درصد این رکوردها با استفاده از دستورهای print() نمایش داده می‌شوند.

سپس با استفاده از sorted(hub_data) مقادیر فاصله را به ترتیب صعودی مرتب می‌کنیم و نتیجه را دوباره در متغیر hub_data قرار می‌دهیم .دستور n = len(hub_data) نیز تعداد کل مقادیر معتبر را مشخص می‌کند .مرتب‌سازی داده‌ها باعث می‌شود بتوانیم صدک‌های موردنظر را از میان مقادیر مرتب‌شده انتخاب کنیم.

در ادامه، صدک‌های ۲۵، ۵۰، ۷۵ و ۹۰ را با استفاده از اندیس‌های محاسبه‌شده از تعداد داده‌ها به دست می‌آوریم .برای مثال، عبارت hub_data[int(0.25 * (n - 1))] مقداری را در موقعیت تقریبی صدک ۲۵ انتخاب می‌کند .به همین ترتیب، صدک‌های دیگر نیز با ضرایب ۰٫۵۰، ۰٫۷۵ و ۰٫۹۰ محاسبه می‌شوند .صدک ۲۵ نشان می‌دهد تقریباً ۲۵ درصد داده‌ها در آن مقدار یا پایین‌تر قرار دارند؛ صدک ۵۰ همان میانه است و صدک‌های ۷۵ و ۹۰ نیز فاصله‌هایی را نشان می‌دهند که تقریباً ۷۵ و ۹۰ درصد رکوردها در آن مقدار یا پایین‌تر قرار گرفته‌اند .در پایان این بخش، مقادیر محاسبه‌شده چاپ می‌شوند .توجه داشته باشید که این روش از انتخاب اندیس استفاده می‌کند و با روش‌های استاندارد درون‌یابی صدک‌ها ممکن است نتایج کمی متفاوتی داشته باشد.

در مرحله‌ی بعد، با دستور Full_data[Full_data["hub_km"] > 300] رکوردهایی را که فاصله‌ی آن‌ها بیشتر از ۳۰۰ کیلومتر است، از جدول اصلی جدا می‌کنیم و در متغیر far_places ذخیره می‌کنیم .سپس با استفاده از far_places["nearest_hub"].value_counts().head(10) مرکز استانی پرتکرار را در میان این رکوردها پیدا می‌کنیم .تابع value_counts() تعداد تکرار هر مقدار در ستون nearest_hub را محاسبه می‌کند و head(10) ده مورد اول را نمایش می‌دهد .این خروجی به شناسایی مرکزهایی کمک می‌کند که تعداد زیادی از رکوردهای دورتر از ۳۰۰ کیلومتر به آن‌ها نسبت داده شده‌اند.

در پایان، با استفاده از far_places["lat"].mean() میانگین عرض جغرافیایی و با far_places["lon"].mean() میانگین طول جغرافیایی رکوردهای دورتر از ۳۰۰ کیلومتر را محاسبه می‌کنیم .این دو مقدار یک تصویر کلی از موقعیت جغرافیایی این رکوردها ارائه می‌دهند؛

خروجی:

```
Number of records over 300 km: 2300
Percentage over 300 km: 0.7740146457032092 %

Percentiles of hub_km:
25th percentile: 5.0
50th percentile: 14.99
75th percentile: 69.98
90th percentile: 120.95

Most common cities for records over 300 km:
nearest_hub
Zahedan        1784
BandarAbbas     515
Ardabil           1

Average coordinates of records over 300 km:
Latitude: 25.950330324347824
Longitude: 60.63325906999999
```

۳ .دو نمودار رسم کنید: یکی هیستوگرام سال ثبت(edit_date) ، که روند رشد پوشش OSM در ایران طی سال‌ها را نشان دهد، و یکی باکس‌پلات km_hub به تفکیک پنج nearest_hub پرتکرار.

کد:

```python
import matplotlib.pyplot as plt


Full_data["edit_date"] = pd.to_datetime(Full_data["edit_date"], errors="coerce")
Full_data["year"] = Full_data["edit_date"].dt.year

plt.figure(figsize=(10, 5))
Full_data["year"].dropna().hist(bins=20, edgecolor="black")
plt.title("OSM Records by Year")
plt.xlabel("Year")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()



top_hubs = Full_data["nearest_hub"].value_counts().head(5).index


hub_data = Full_data[Full_data["nearest_hub"].isin(top_hubs)]

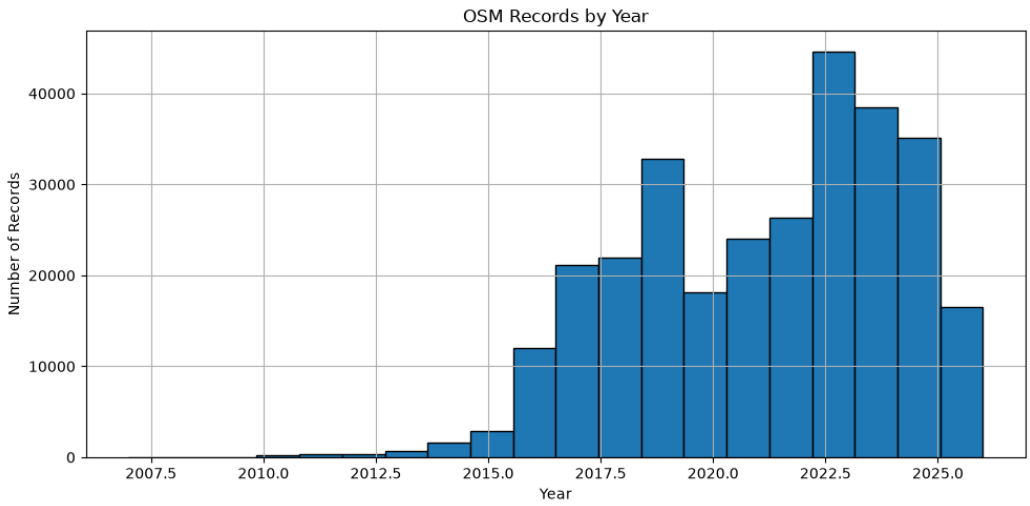
plt.figure(figsize=(10, 5))
hub_data.boxplot(column="hub_km", by="nearest_hub")
plt.title("Distance from Nearest Hub for Top 5 Hubs")
plt.suptitle("")
plt.xlabel("Nearest Hub")
plt.ylabel("Distance (km)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

با اســتفاده از pd.to_datetime(Full_data["edit_date"], errors="coerce") ســتون edit_date را به قالب تاریخ و زمان تبدیل می‌کنیم .پارامتر errors="coerce" باعث می‌شــود مقادیر نامعتبر به NaT تبدیل شــوند .در مرحله‌ی بعد، با دســتور Full_data["edit_date"].dt.year سال ثبت هر رکورد را استخراج می‌کنیم و در ستون جدید year قرار می‌دهیم.

برای رسم نمودار اول، با استفاده از plt.figure(figsize=(10, 5)) اندازه‌ی شکل را مشخص می‌کنیم .سپس دستور Full_data["year"].dropna().hist(bins=20, edgecolor="black") هیستوگرام سال ثبت رکوردها را رسم می‌کند .تابع dropna() مقادیر خالی را حذف می‌کند، پارامتر bins=20 داده‌ها را به ۲۰ بازه تقسیم می‌کند و edgecolor="black" حاشیه‌ی ستون‌های نمودار را مشکی قرار می‌دهد .با استفاده از plt.title() عنوان نمودار، با plt.xlabel() عنوان محور افقی و با plt.ylabel() عنوان محور عمودی را مشخص می‌کنیم .دستور plt.tight_layout() فاصله‌ی اجزای نمودار را تنظیم می‌کند تا نوشته‌ها به‌خوبی نمایش داده شوند و در نهایت plt.show() نمودار را نشان می‌دهد .این هیستوگرام توزیع تعداد رکوردها بر اساس سال ثبت را نمایش می‌دهد و می‌تواند برای بررسی روند رشد داده‌های OSM در طول زمان استفاده شود؛ البته برای تفسیر دقیق رشد پوشش، باید توجه داشت که این نمودار تعداد رکوردهای موجود به تفکیک سال ثبت را نشان می‌دهد، نه لزوماً تعداد رکوردهای جدید اضافه‌شده در هر سال.در مرحله‌ی بعد، پنج مقدار پرتکرار ستون nearest_hub را با دستور Full_data["nearest_hub"].value_counts().head(5).index استخراج می‌کنیم و در متغیر top_hubs قرار می‌دهیم .تابع value_counts() تعداد رکوردهای مربوط به هر مرکز را محاسبه می‌کند، head(5) پنج مورد پرتکرار را انتخاب می‌کند و index نام این مراکز را برمی‌گرداند .سپس با استفاده از Full_data[Full_data["nearest_hub"].isin(top_hubs)] فقط رکوردهایی را انتخاب می‌کنیم که مرکز آن‌ها یکی از این پنج مورد باشد و نتیجه را در متغیر hub_data ذخیره می‌کنیم.

برای رسم نمودار دوم، ابتدا با plt.figure(figsize=(10, 5)) اندازه‌ی نمودار را مشخص می‌کنیم .سپس دستور hub_data.boxplot(column="hub_km", by="nearest_hub") باکس‌پلات فاصله‌ی رکوردها تا مرکز مربوطه را برای پنج مرکز پرتکرار رسم می‌کند .در این نمودار، خط داخل جعبه میانه‌ی فاصله‌ها را نشان می‌دهد، محدوده‌ی جعبه از صدک ۲۵ تا صدک ۷۵ را نمایش می‌دهد و خطوط امتدادی، پراکندگی داده‌ها را طبق قواعد باکس‌پلات مشخص می‌کنند .مقادیر پرت نیز در صورت وجود به‌صورت جداگانه نمایش داده می‌شوند .دستور plt.title() عنوان نمودار را تعیین می‌کند و plt.suptitle("") عنوان کلی پیش‌فرضی را که پانداس برای گروه‌بندی اضافه می‌کند، حذف می‌کند .سپس با plt.xlabel() و plt.ylabel() عنوان محورهای افقی و عمودی را مشخص می‌کنیم .دستور plt.xticks(rotation=30) نام مراکز روی محور افقی را ۳۰ درجه می‌چرخاند تا خواندن آن‌ها آسان‌تر شود .در پایان، با plt.tight_layout() چیدمان نمودار تنظیم و با plt.show() نمایش داده می‌شود.

هیستوگرام تاریخ ادیت:

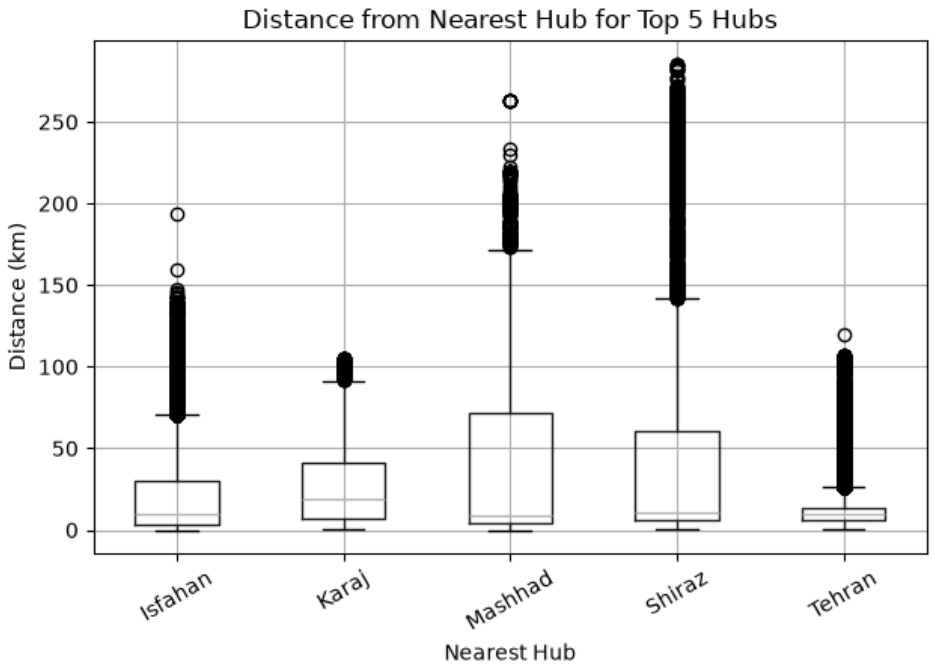


تحلیل: در سال‌های ابتدایی، تقریباً از ۲۰۱۰ تا ۲۰۱۳، تعداد رکوردها بسیار کم است .از حدود سال ۲۰۱۴ به بعد، تعداد رکوردها افزایش پیدا می‌کند و در سال‌های ۲۰۱۶ تا ۲۰۱۹ رشد قابل‌توجهی مشاهده می‌شود .این روند می‌تواند نشان‌دهنده‌ی افزایش فعالیت کاربران و گسترش داده‌های ثبت‌شده در OSMباشد.

در سال ۲۰۱۹ تعداد رکوردها به حدود ۳۳ هزار می‌رسد، اما در سال ۲۰۲۰ کاهش پیدا می‌کند .پس از آن، تعداد رکوردها دوباره افزایش می‌یابد و در سال‌های ۲۰۲۲ و ۲۰۲۳ رشد چشمگیری دیده می‌شود .بیشترین مقدار نمودار مربوط به حدود سال ۲۰۲۳ است که تعداد رکوردها به تقریباً ۴۵ هزار می‌رسد.

پس از سال ۲۰۲۳، تعداد رکوردها کاهش می‌یابد؛ به‌طوری‌که در سال ۲۰۲۴ حدود ۳۸ هزار، در سال ۲۰۲۵ حدود ۳۵ هزار و در سال ۲۰۲۶ حدود ۱۶ هزار رکورد مشاهده می‌شود.

باکس پلات nearest_hub :



تحلیل:

- خط داخل جعبه :میانه‌ی فاصله‌ها را نشان می‌دهد؛ یعنی نیمی از رکوردها فاصله‌ای کمتر و نیمی فاصله‌ای بیشتر از این مقدار دارند.
- جعبه :محدوده‌ی بین صدک ۲۵ و صدک ۷۵ را نشان می‌دهد .هرچه جعبه بلندتر باشد، پراکندگی فاصله‌ها بیشتر است.
- خطوط امتدادی جعبه :محدوده‌ی داده‌ها را طبق قواعد باکس‌پلات نشان می‌دهند.
- نقاط سیاه خارج از خطوط :مقادیر پرت هستند؛ یعنی فاصله‌هایی که نسبت به توزیع اصلی داده‌ها غیرمعمول محسوب می‌شوند.

۱ .اصفهان (Isfahan): میانه‌ی فاصله‌ها نسبتاً پایین است، اما تعدادی مقدار پرت تا حدود ۱۹۰ کیلومتر مشاهده می‌شود . این موضوع نشان می‌دهد بیشتر رکوردها به مرکز نزدیک‌اند، ولی تعدادی فاصله‌ی زیادی دارند.

۲ .کرج (Karaj): میانه‌ی فاصله‌ها از اصفهان بیشتر است و بیشتر داده‌ها در محدوده‌ی پایین‌تری قرار دارند .مقادیر پرت نیز تا حدود ۱۰۵ کیلومتر دیده می‌شوند.

۳ .مشهد (Mashhad): جعبه ارتفاع زیادی دارد که نشان‌دهنده‌ی پراکندگی قابل‌توجه فاصله‌هاست .همچنین تعدادی مقدار پرت تا حدود ۲۶۰ کیلومتر مشاهده می‌شود.

۴ .شیراز (Shiraz): تعداد زیادی مقدار پرت وجود دارد و برخی فاصله‌ها به حدود ۲۸۰ کیلومتر می‌رسند .گستردگی جعبه نیز نشان می‌دهد فاصله‌ها در این گروه پراکندگی زیادی دارند.

۵ .تهران (Tehran): جعبه نسبتاً کوچک است و میانه‌ی فاصله‌ها پایین قرار دارد .بااین‌حال، مقادیر پرت تا حدود ۱۲۰ کیلومتر مشاهده می‌شوند؛ یعنی با وجود نزدیک بودن بیشتر رکوردها، برخی نقاط از مرکز فاصله‌ی زیادی دارند.

۴ .یک دیتاکیوب با سه بُعد nearest_hub ، main_key و سال edit_date بسازید که مقدار هر سلولش تعداد رکورد باشد. سپس روی همین دیتاکیوب سه عملیات زیر را انجام دهید و برای هرکدام یک جمله بنویسید که چه سوالی را جواب داد:

- **یک up-roll: بُعد سال را حذف کنید و فقط جمع کل رکوردها را بر اساس شهر و دسته نگه دارید.**
- **یک drill-down: برای یک شهر و دسته‌ی مشخص) مثلاً leisure در تهران(، به سطح جزئی‌تر main_value بروید و ببینید این دسته از چه زیردسته‌هایی تشکیل شده.**
- **یک Selection به سبک فصل ۳: فقط رکوردهای تهران را نگه دارید و توزیع دسته‌ها را درون همان یک شهر گزارش دهید.**

کد دیتا کیوب اول:

```
import pandas as pd

Full_data["edit_date"] = pd.to_datetime(
    Full_data["edit_date"], errors="coerce"
)

Full_data["year"] = Full_data["edit_date"].dt.year

data_cube = pd.pivot_table(
    Full_data,
    index=["year", "main_key"],
    columns="nearest_hub",
    aggfunc="size",
    fill_value=0
)


display(data_cube)
```

با اســـتفاده از pd.to_datetime(Full_data["edit_date"], errors="coerce")ســـتون edit_dateرا به قالب تاریخ و زمان تبدیل می‌کنیم .پارامتر errors="coerce"باعث می‌شـــود مقادیر نامعتبر به NaTتبدیل شـــوند .در مرحله‌ی بعد، با دســـتور Full_data["edit_date"].dt.yearسال هر رکورد را استخراج می‌کنیم و در ستون جدید yearقرار می‌دهیم.

ســپس با اســتفاده از تابع pd.pivot_table()دیتاکیوب را ایجاد می‌کنیم .این تابع داده‌ها را بر اســاس ابعاد مشــخص‌شــده گروه‌بندی می‌کند و نتایج را در قالب یک جدول محوری نمایش می‌دهد .در این کد، آرگومان index=["year", "main_key"] مشخص می‌کند که سطرهای جدول بر اساس ترکیب سال و main_keyساخته شوند .آرگومان columns="nearest_hub" نیز مقادیر مختلف ستون nearest_hubرا به ستون‌های جداگانه در جدول تبدیل می‌کند.

آرگومان aggfunc="size"مشـــخص می‌کند که مقدار هر ســـلول برابر با تعداد رکوردهای مربوط به ترکیب آن ســـال، آن main_keyو آن nearest_hubباشـــد .برای مثال، اگر در یک ســـال مشـــخص، ۱۲۰ رکورد با main_keyبرابر amenityو nearest_hubبرابر تهران وجود داشته باشد، مقدار سلول مربوط به این ترکیب برابر با ۱۲۰ خواهد بود.

همچنین، آرگومان fill_value=0باعث می‌شـــود ســـلول‌هایی که هیچ رکوردی برای ترکیب مربوط به آن‌ها وجود ندارد، به‌جای مقدار خالی با عدد صفر پر شوند .در نتیجه، تمام ترکیب‌های نمایش‌داده‌شده در جدول دارای مقدار عددی خواهند بود.

در پایان، دستور display(data_cube)دیتاکیوب ساخته‌شده را نمایش می‌دهد تا بتوانیم تعداد رکوردها را بر اساس سه بُعد موردنظر بررسی کنیم.

خروجی:

	nearest_hub	Ahvaz	Arak	Ardabil	BandarAbbas	Birjand	Bojnord	Bushehr	Gorgan	Hamadan	Ilam	...	Semnan	Shahrekord	Shiraz	Tabriz	Tehran	Urmia
year	main_key																	
2007	amenity	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2008	amenity	0	0	0	0	0	0	0	0	0	0	...	1	0	0	0	0	0
	leisure	0	0	0	2	0	0	0	0	0	0	...	0	0	0	0	0	0
	tourism	0	0	0	1	0	0	0	0	0	0	...	0	0	0	0	0	0
2009	amenity	0	0	0	0	0	0	0	0	0	0	...	0	0	1	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2026	healthcare	9	5	3	5	1	2	1	4	2	0	...	0	2	15	7	65	1
	leisure	44	15	10	94	16	10	27	19	21	10	...	4	35	98	25	385	18
	office	123	23	21	66	9	11	27	19	11	7	...	2	10	156	43	892	29
	shop	392	50	69	110	17	20	32	74	34	20	...	9	19	291	121	1615	62
	tourism	11	8	11	24	62	2	13	1	6	2	...	0	2	24	6	68	4

کد Roll_up :

```
roll_up = pd.pivot_table(
    Full_data,
    index=["main_key"],
    columns="nearest_hub",
    aggfunc="size",
    fill_value=0
)


display(roll_up)
```

✓ 0.0s Python

nearest_hub	Ahvaz	Arak	Ardabil	BandarAbbas	Birjand	Bojnord	Bushehr	Gorgan	Hamadan	Ilam	...	Semnan	Shahrekord	Shiraz	Tabriz	Tehran	Urmia
main_key																	
amenity	4090	4616	3015	4783	2218	1989	2300	2252	2694	1183	...	1062	2310	8607	6505	26145	3314
craft	98	138	72	120	55	60	42	56	45	19	...	26	40	242	172	792	72
healthcare	54	25	34	41	42	19	15	19	32	5	...	11	19	134	76	778	35
leisure	1667	1310	1466	1297	902	464	942	512	612	380	...	308	641	2731	1329	9802	682
office	795	731	583	707	404	312	326	387	390	283	...	226	457	1807	1111	6156	623
shop	1956	2280	1256	2074	656	813	824	1029	850	328	...	434	651	7210	2712	15768	1380
tourism	383	422	385	550	235	123	170	214	188	65	...	127	599	675	482	1289	206

در دیتاکیوب قبلی، سه بُعد داشتیم year ،: main_key و nearest_hub. در کد جدید، بُعد year را حذف کرده‌ایم و فقط main_key و nearest_hub را نگه داشته‌ایم .بنابراین، رکوردهای تمام سال‌ها با هم تجمیع می‌شوند و تعداد کل رکوردها برای هر ترکیب از main_key و nearest_hub محاسبه می‌شود.

- Roll-up: کاهش جزئیات و تجمیع داده‌ها؛ مثلاً حذف بُعد سال برای محاسبه‌ی تعداد کل رکوردها در همه‌ی سال‌ها.

کد Drill_down :

```python
drill_down = pd.pivot_table(
    Full_data,
    index=["main_key", "main_value"],
    columns="nearest_hub",
    aggfunc="size",
    fill_value=0
)

display(drill_down)
```

✓ 0.1s

	nearest_hub	Ahvaz	Arak	Ardabil	BandarAbbas	Birjand	Bojnord	Bushehr	Gorgan	Hamadan	Ilam	...	Semnan	Shahrekord	Shiraz	Tabriz
main_key	main_value															
amenity	Company	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
	Greenhouse	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
	Investment	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
	Kaveh Parts and Seat Sales	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
	Physiotherapy	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
tourism	yes	0	18	4	2	0	0	1	3	0	0	...	0	9	5	4
	zoo	1	0	1	3	0	1	0	1	1	0	...	0	0	3	1
	محل عبادت	0	0	0	0	0	1	0	0	0	0	...	0	0	0	0
	میدان	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
	میدان ساعت	0	0	0	0	0	0	0	0	0	0	...	0	0	0	1

در این کد، عملیات Drill-down را انجام می‌دهیم؛ یعنی سطح جزئیات داده‌ها را افزایش می‌دهیم .در دیتاکیوب قبلی، داده‌ها بر اساس main_key و nearest_hub گروه‌بندی شده بودند؛ اما در این کد، بُعد main_value را نیز اضافه می‌کنیم تا بتوانیم جزئیات بیشتری از رکوردها را بررسی کنیم.

- Drill-down : افزایش جزئیات؛ مثلاً اضافه کردن بُعد سال به جدول برای مشاهده‌ی تعداد رکوردها در هر سال.

کد selection :

```python
selection = data_cube["Tehran"].reset_index()
count = data_cube["Tehran"].groupby(level="main_key").sum()

display(selection)
print("Total numbers of per main_key for Tehran: ")
print(count.to_string())
```

	main_key	main_value	Tehran
0	amenity	Company	1
1	amenity	Greenhouse	1
2	amenity	Investment	1
3	amenity	Kaveh Parts and Seat Sales	0
4	amenity	Physiotherapy	1
...	...	...	...
1203	tourism	yes	3
1204	tourism	zoo	9
1205	tourism	محل عبادت	0
1206	tourism	میدان	0
1207	tourism	میدان ساعت	0

1208 rows × 3 columns

```
Total numbers of per main_key for Tehran:
main_key
amenity       26145
craft           792
healthcare      778
leisure        9802
office         6156
shop          15768
tourism        1289
```

در این کد، عملیات Selectionرا روی دیتاکیوب انجام می‌دهیم؛ یعنی داده‌های مربوط به یک مرکز استانی مشخص، در اینجا تهران، را انتخاب می‌کنیم تا بتوانیم اطلاعات آن را جداگانه بررسی کنیم.

در خط اول، دستور data_cube["Tehran"].reset_index()ستون مربوط به تهران را از دیتاکیوب استخراج می‌کند و با استفاده از reset_index()، شاخص‌های جدول یعنی main_keyو main_valueرا به ستون‌های معمولی تبدیل می‌کند . نتیجه در متغیر selectionذخیره می‌شود .سپس با دستور display(selection)، اطلاعات مربوط به تهران را به تفکیک main_keyو main_valueنمایش می‌دهیم.

در مرحله‌ی بعد، دستور data_cube["Tehran"].groupby(level="main_key").sum()داده‌های مربوط به تهران را بر اساس main_keyگروه‌بندی می‌کند و با استفاده از sum()تعداد رکوردهای مربوط به هر کلید اصلی را در تمام مقادیر main_value جمع می‌زند .نتیجه در متغیر countذخیره می‌شود.

پ . ن : در عملیات drill down و selection بعد سال درنظر نگرفته شده است